When Detection Fails, Fixing Doesn't Matter: Composing LLMs for Automated Security Patching

Can LLMs replace static analysis for vulnerability detection and repair?
An empirical evaluation across 7 models, 49 pipeline combinations, and 23 CWE categories

Timothy Liakh
Rensselaer Polytechnic Institute
Advisor: Konstantin Kuzmin
Summer 2026

Repository: github.com/Vitisadi/SecurePatch

Abstract

Static analysis has long been the default answer to the question of how you catch security vulnerabilities before they ship. Tools like Semgrep and CodeQL are fast, deterministic, and cheap to run. But they flag patterns, not intent — and they cannot generate a fix. Large language models offer a different trade-off: they read code semantically, can explain why something is dangerous, and can propose a patch in the same step. Whether that reasoning is actually reliable enough to matter, and for which vulnerability classes, is less settled than the marketing suggests.

This project evaluates seven LLMs as both detectors and fixers across 56 cases spanning 23 CWE categories, drawn from CWEval (44 cases), hand-seeded examples (6), and published literature (6). Detection and repair are treated as separate, composable stages. A detector scans code and produces findings; a fixer receives those findings and returns a corrected file. Running all combinations produces a 7×7 matrix of 49 pipelines, each scored on fix rate, regression rate, F1, and cost by a deterministic oracle.

Solo models peak at 75 % F1 (GPT-5.5). When each fixer is given a perfect detection (zero false positives, all 56 bugs correctly identified), Opus fixes 76 % of cases. The gap between that ceiling and real pipeline performance traces almost entirely to false positives at the detection step, not to any weakness in the fixers themselves. Across the heatmap, which fixer you choose matters far less than which detector feeds it.

Cost varies by three orders of magnitude. Haiku paired with GPT-4.1-mini reaches 68 % F1 for $0.11 across all 56 cases; the GPT-5.5 self-pipeline costs $7.10 for 75 % F1. Ensemble detection via majority vote across frontier models pushes detection F1 to 93 %, but the gain over the best solo detector is roughly 2 percentage points. Modest, and not free.

The finding that runs through everything: false-positive rate in detection sets the ceiling of the pipeline. Upgrading the fixer without first cleaning up detection produces rapidly diminishing returns.

Contents

1 Introduction

Static analysis security testing has been the default tool for catching vulnerabilities in production code for over two decades. Semgrep, CodeQL, and Bandit operate on fixed rule sets: fast, deterministic, and guaranteed to catch anything their rules describe. That guarantee is also the limit. A rule that does not exist catches nothing. Writing rules requires security expertise most development teams do not have. And even when that expertise exists, the rules age. New vulnerability patterns emerge faster than rule sets are updated. More practically: none of these tools fix what they find. A developer still has to read the alert, understand the root cause, and write the patch.

Large language models enter this space differently. They are not bound by explicit rules; they reason about code semantically and can generalize across vulnerability classes without being programmed for each one. They can generate a patch in the same pass. Whether that reasoning is reliable enough to replace or meaningfully augment static analysis, or which models perform well at detection versus repair, has not been studied systematically across a controlled benchmark. That gap is what this project addresses.

This project measures exactly that. Seven LLMs are evaluated as detectors and as fixers, independently, across 56 real vulnerabilities spanning 23 CWE categories. Detection and repair are treated as composable stages: a detector produces findings, and a fixer receives those findings and generates a patch. Every combination of the seven detectors and seven fixers is run, producing 49 distinct pipelines. Each pipeline is judged by a deterministic oracle on fix rate, regression rate, F1, and cost.

The central question is:

RQ1. *Can LLMs match or exceed static analysis for vulnerability detection, and does composing a strong detector with a strong fixer produce better security outcomes than either model working alone?*

RQ2. *Does running multiple detectors and taking a majority vote meaningfully improve detection quality, and is the added cost justified by the gain in F1?*

1.1 Contributions

1. **A controlled vulnerability benchmark.** 56 cases drawn from CWEval (44), hand-seeded examples (6), and published literature (6), covering 23 CWE categories. Each case has a ground-truth vulnerable file, a known fix, and a deterministic oracle that distinguishes a correct patch from a regression or a no-op.
2. **A reproducible evaluation harness.** The `CachedFindingDetector` replays pre-computed detection outputs identically across all fixer columns, ensuring that differences in pipeline F1 reflect fixer behavior rather than detection variance. All results are stored as versioned JSONL

files and can be regenerated from the repository.

3. **A full 7×7 pipeline heatmap.** Every combination of seven detectors and seven fixers is evaluated, producing 49 F1 scores on the same 56-case benchmark. This is the primary empirical contribution of the project.

4. **A ground-truth fixer experiment.** Each fixer is given a perfect detection with zero false positives and 100 % recall, isolating pure repair capability from detection noise. The gap between this ceiling and real pipeline F1 quantifies how much detection quality costs each fixer.

5. **An ensemble detection study.** Majority-vote combinations of multiple detectors are evaluated on F1, recall, and false-positive count. The best configuration, a frontier majority vote across four models, reaches 93 % F1 with 100 % recall.

1.2 Roadmap

The paper begins by grounding this work relative to existing research on LLM-based vulnerability detection and automated program repair. It then describes how the benchmark was constructed and how the evaluation pipeline works. The results follow in order of increasing complexity: solo model performance first, then the full composition matrix, then cost and ensemble analysis. The paper closes with a discussion of what the findings mean in practice, the limits of the current study, and directions for follow-on work.

2 Background and Related Work

2.1 LLM-Based Vulnerability Detection

Detecting security vulnerabilities in source code has traditionally been the domain of static analysis. Tools like Semgrep and CodeQL match code against hand-written rules, offering speed and determinism at the cost of coverage: they catch only what their rules describe. Machine learning approaches tried to close that gap, with models trained on historical vulnerability data to predict whether a function or line was dangerous Fu and Tantithamthavorn [2022], Thapa et al. [2022], Li et al. [2023]. These models improved generalization but introduced a new problem: they produce false positives at rates that make triage expensive in practice.

Large language models enter this space differently. Rather than classifying against learned patterns, they read code and reason about what it does. Pearce et al. showed that LLMs could identify security weaknesses in code generated by GitHub Copilot Pearce et al. [2022], and later demonstrated that models could detect and explain vulnerabilities in hand-crafted scenarios without any task-specific fine-tuning Pearce et al. [2023]. Meta's CyberSecEval benchmark evaluated frontier models on a range of security tasks and found substantial variance across CWE categories Bhatt et al. [2023]. What these studies share is a focus on a single model acting alone. This project asks a different

question: what happens when you pick one model to detect and a different model to fix?

2.2 Automated Program Repair

Automated program repair has a long history independent of security. The core idea is that given a failing test and a buggy program, a system should be able to produce a patch without human intervention. Early approaches used genetic algorithms and constraint-based synthesis. LLMs changed what was possible. Xia et al. found that pre-trained code models could generate correct patches at rates that surpassed prior symbolic methods on standard benchmarks Xia et al. [2023], and Jiang et al. showed that model scale was a reliable predictor of repair success Jiang et al. [2023]. Sobania et al. evaluated ChatGPT specifically on the QuixBugs benchmark and found it competitive with dedicated APR tools on simple bugs Sobania et al. [2023].

The security repair problem is harder than general APR. A patch that makes a test pass is not necessarily a secure patch: it may suppress the symptom without removing the vulnerability, or introduce a regression elsewhere. Pearce et al. examined this directly, finding that LLMs often generated plausible-looking but insecure fixes when prompted without explicit security guidance Pearce et al. [2023]. This project addresses that by using a deterministic oracle that checks both fix success and regression, and by separating the detection signal from the repair step so the fixer receives an explicit vulnerability description rather than just a failing test.

2.3 Existing Benchmarks

SWE-bench Jimenez et al. [2024] is the closest large-scale benchmark to this project. It draws real GitHub issues from Python repositories and validates patches against the project's own test suite. The focus there is model capability on general software tasks. SecurityEval Siddiq and Santos [2022] targets security specifically, using 130 code generation prompts mapped to CWE categories to test whether models produce vulnerable output. CyberSecEval takes a broader view, covering prompt injection, insecure code generation, and vulnerability identification across multiple languages Bhatt et al. [2023].

None of these benchmarks treat detection and repair as a composed pipeline. They either test one capability at a time or evaluate a single model end to end. The 7×7 design here is, to our knowledge, the first systematic study of cross-model composition for security vulnerability repair.

2.4 Where This Project Fits

This project is closest in spirit to SWE-bench but more controlled in design. Every case has a known-vulnerable commit, a verified fix, and a deterministic oracle that does not rely on an LLM judge. The central contribution is not a new model or a new benchmark in isolation, but a methodology for measuring what each stage of a detect-then-fix pipeline actually contributes to the final outcome.

3 Benchmark Design

3.1 Case Selection and Dataset Composition

The benchmark contains 56 cases drawn from three sources. The majority (44) come from CWEval Peng et al. [2025], an open collection of vulnerability scenarios designed for evaluating LLM security capabilities. Six additional cases were hand-seeded: written from scratch to cover CWE categories underrepresented in CWEval or to test boundary conditions that naturally occurring examples rarely produce. The remaining six come from published vulnerability disclosures and literature examples where a real fix commit is publicly available.

Every case provides the same inputs to the pipeline: a source file containing a vulnerability, the CWE category it belongs to, and a reference fix. Cases were selected to have self-contained vulnerabilities, meaning the flaw and its fix are localized to a single file. This keeps the evaluation clean: a fixer that modifies the right file either produces a correct patch or it does not, and the oracle can verify this without reasoning about cross-file dependencies.

Each case is also tagged with an obscurity tier that describes how hard the vulnerability is to find without semantic reasoning. Ten cases are *syntactic*: the flaw follows a pattern a regex rule could catch. Forty-five are *local-semantic*: the vulnerability requires following data flow within the file to understand why the code is dangerous. One is *cross-function*: the source and sink are in different functions, making the flaw invisible without call-graph reasoning. This distribution intentionally weights toward cases that require genuine semantic understanding, the class where LLMs should have an advantage over static analysis.

3.2 CWE Coverage

The 56 cases span 23 distinct CWE categories. Table 1 lists each CWE present in the benchmark along with its case count. The distribution is intentionally varied: no single category dominates, which prevents a model from scoring well by specializing in one vulnerability class. The most represented categories reflect the types of flaws most commonly found in real code, including injection vulnerabilities, improper input validation, and cryptographic weaknesses.

Table 1: CWE categories covered in the benchmark (56 cases total, 23 CWEs).

CWE ID	Description	Cases
CWE-327	Use of Broken or Risky Cryptographic Algorithm	6
CWE-22	Path Traversal	4
CWE-326	Inadequate Encryption Strength	4
CWE-918	Server-Side Request Forgery (SSRF)	4
CWE-78	OS Command Injection	4
CWE-502	Deserialization of Untrusted Data	3
CWE-79	Cross-site Scripting (XSS)	3
CWE-89	SQL Injection	3
CWE-95	Code Injection (Eval)	3
CWE-113	HTTP Response Splitting	2
CWE-117	Improper Output Neutralization for Logs	2
CWE-1333	Inefficient Regular Expression (ReDoS)	2
CWE-329	Generation of Predictable IV with CBC	2
CWE-347	Improper Verification of Cryptographic Signature	2
CWE-400	Uncontrolled Resource Consumption	2
CWE-643	XPath Injection	2
CWE-943	NoSQL Injection	2
CWE-20	Improper Input Validation	1
CWE-330	Use of Insufficiently Random Values	1
CWE-377	Insecure Temporary File	1
CWE-732	Incorrect Permission Assignment	1
CWE-760	Use of a One-Way Hash with Predictable Salt	1
CWE-798	Use of Hard-coded Credentials	1
Total		56

3.3 Evaluation Oracle

Each case is judged by a deterministic oracle rather than an LLM reviewer. The oracle runs in a Docker container to ensure consistent execution across all pipeline combinations. Given the original vulnerable file and the fixer's output, it checks three conditions in order: whether the patched file compiles or parses correctly, whether the targeted vulnerability is no longer present according to a rule matched to the case's CWE, and whether the fix introduced any regression by running a suite of behavioral checks against the original file's expected outputs.

A case is counted as a fix only if all three conditions pass. A patch that removes the vulnerability but breaks existing behavior is recorded as a regression, not a fix. A patch that makes no change, or changes code unrelated to the vulnerability, is recorded as a no-op. This three-way verdict is what

allows fix rate and regression rate to be reported independently rather than collapsed into a single success metric.

4 Pipeline Architecture and Methodology

4.1 Detect → Enrich → Fix Pipeline

The harness runs two distinct experiments. Experiment 1 measures detection: a model reads the source file and returns findings; those findings are matched against ground-truth bug locations to produce a recall score and a false positive count. Experiment 2 measures fix-and-verify: the fixer receives the bug location and description from Experiment 1 and returns a patched file, which the oracle then evaluates for correctness and regression. F1 is computed across both experiments together, penalizing false positives from detection and missed fixes from repair.

Each pipeline run has two stages. In the first, a detector reads the vulnerable source file and returns a list of findings. Each finding carries a vulnerability type, a line number, and a natural-language description of why the code is dangerous. In the second stage, a fixer receives the full source file together with those findings as structured context, and returns a corrected version of the file.

The fixer is prompted to act as a precise application-security engineer: fix the specific vulnerability identified, preserve all other behavior, and change only what is needed. It is instructed not to add explanations, tests, or unrelated refactors. The response is a complete rewritten file rather than a diff. Whole-file replacement was chosen over patch-based approaches because it avoids fuzzy context alignment failures: the verifier always receives a syntactically complete file regardless of how large the fix is.

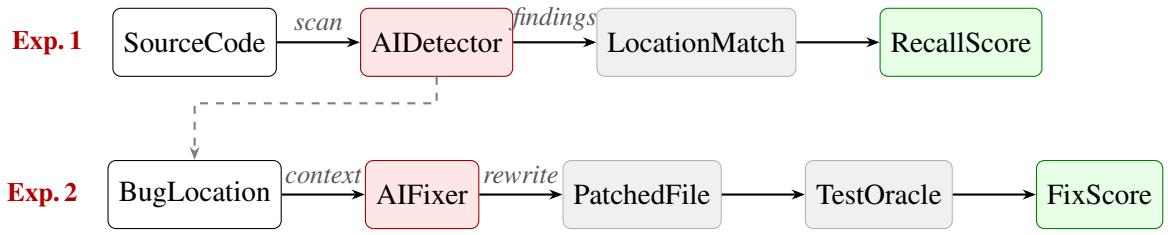

Figure 1: End-to-end pipeline architecture. Experiment 1 runs the detector and scores recall. Experiment 2 feeds the detector's findings to the fixer, then runs the oracle to produce the fix verdict. The dashed arrow shows findings flowing from detection into the fix prompt.

4.2 CachedFindingDetector

A key methodological choice is that all 49 pipeline combinations share the same detection outputs for a given detector. Detection is run once per model per case and the results are stored in a versioned JSONL file. The `CachedFindingDetector` replays those stored findings identically for every

fixer column in that detector's row.

This matters for fairness. If detection were re-run live for each detector-fixer pair, any variance in the model's output across calls could cause two fixers in the same row to receive different findings for the same case. The cached approach eliminates that source of noise: differences in F1 across a detector row reflect only fixer behavior, not detection variance.

Each cached record stores the exact findings the detector produced, including matched bug findings with their type, line number, and description, as well as any false positives. During a fix run, the `CachedFindingDetector` serves the stored initial findings to the fixer prompt, then delegates post-fix re-scanning to the regex detector or the CWEval Docker oracle, whichever applies to that case type.

4.3 Models Under Test

Table 2: Models evaluated as detectors and fixers, with solo detection performance and cost per 56-case run. Regex and Semgrep are included as SAST baselines (detector only).

Model	Role	Recall	Prec.	F1	Cost
Sonnet 4.6	Det + Fix	100%	79%	88%	$0.330
Opus 4.8	Det + Fix	91%	78%	84%	$0.731
Haiku 4.5	Det + Fix	91%	72%	80%	$0.109
GPT-5.5	Det + Fix	93%	91%	92%	$1.841
GPT-4.1-mini	Det + Fix	70%	85%	76%	$0.028
Gemini 2.5 Flash	Det + Fix	91%	86%	89%	$0.047
Ollama 7b	Det + Fix	68%	56%	61%	$0.000
Regex (custom)	Detector	34%	86%	49%	$0.000
Semgrep OSS	Detector	20%	95%	32%	$0.000

4.4 Metrics

The primary metric is F1, which accounts for both the detector's tendency to produce false positives and the fixer's ability to produce correct patches:

$$F1 = \frac{2 \cdot P \cdot R}{P + R} \qquad \text{where} \qquad P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

TP is the number of cases correctly fixed without introducing a regression. FP is the number of detector false positives across all 56 cases. FN is $56 - \mathrm{TP}$, the cases where the vulnerability was not removed. Precision P captures the cost of false alarms from detection; recall R captures the cost of

missed fixes. F1 penalizes both equally, making it a single number that reflects the full pipeline's usefulness.

Fix rate and regression rate are reported separately. Fix rate is the fraction of the 56 cases where the fixer produced a correct patch given the detector's output. Regression rate is the fraction of fix attempts where the patch removed the vulnerability but broke existing behavior. A high fix rate with a high regression rate indicates a fixer that is too aggressive.

Cost is measured in USD per 56-case run, computed from the actual token counts recorded by the provider during each run using published price cards. Fixes per dollar is derived from this and gives a practical efficiency metric for comparing pipelines at equal quality.

4.5 Experimental Controls and v2 Methodology

All 49 pipeline combinations run on the same 56 cases with the same cached detection outputs, the same fixer prompt template, and the same oracle. The only variable is which model fills each role.

Two post-hoc corrections were applied to produce the final v2 results. First, the Haiku fixer was found to produce outputs with a JSON fence formatting bug that caused verdicts to be mis-parsed in earlier runs. Verdicts for all Haiku fixer cells were re-derived from the stored raw outputs using the corrected parser. All reported numbers reflect these corrections. The raw JSONL outputs are preserved in the repository.

5 Results

5.1 Solo Pipeline Performance

Table 3 shows results for the diagonal of the heatmap, where the same model acts as both detector and fixer. GPT-5.5 achieves the highest F1 at 75 %, despite a fix rate of only 64 %, because its detection produces just 5 false positives across 56 cases. Opus posts the highest fix rate at 66 % but reaches only 70 % F1 because its detector generates 13 false positives. This divergence between fix rate and F1 is the first signal that detection quality is doing more work than fixing capability.

Ollama is the clear outlier. At 23 % fix rate and 28 % F1, with 35 regressions, it is not competitive with any paid model for this task. The gap between Ollama and the next-weakest model (Gemini at 50 % F1) is larger than the gap between Gemini and the best model.

Table 3: Solo pipeline results (detector = fixer, diagonal of heatmap). Cost is in USD for all 56 cases.

Model	Fix Rate	F1	Regressions	Cost ($)	Fixes/$
GPT-5.5	64%	75%	11	1.84	20
Opus 4.8	66%	70%	10	0.73	51
Sonnet 4.6	62%	67%	18	0.33	106
Haiku 4.5	59%	62%	13	0.11	306
GPT-4.1-mini	54%	63%	13	0.03	1071
Gemini 2.5	50%	56%	23	0.05	596
Ollama 7b	23%	28%	35	0.00	free

5.2 Full 7×7 Pipeline Heatmap

Figure 2 shows F1 for all 49 detector-fixer combinations. The row structure is the dominant pattern: pipelines in the GPT-5.5 detector row reach 61–75 % F1 regardless of which fixer they use, while pipelines in the Ollama detector row top out at 50 % even with the strongest fixers. The column structure is weaker. Switching from the worst fixer (Ollama, 28–42 %) to the best in a given row (typically Opus or GPT-5.5) adds at most 15 percentage points, while switching from the worst detector row to the best adds up to 33 points.

The GPT-5.5 detector row peaks at 75 % F1 (GPT-5.5 fix) and 75 % (GPT-4.1-mini fix), and never falls below 61 % (Gemini fix). The Haiku detector row is the surprise: despite Haiku being a budget model, its row reaches 71 % F1 (Opus fix) and outperforms the Sonnet and Opus detector rows in several columns, because its detection recall is high at 94 % with only 16 false positives.

Det↓/Fix→	Sonnet	Opus	Haiku	GPT-5.5	Mini	Gemini	Ollama
Sonnet	67%	69%	61%	70%	67%	58%	35%
Opus	68%	70%	65%	70%	65%	58%	40%
Haiku	70%	71%	62%	69%	68%	60%	42%
GPT-5.5	72%	75%	74%	75%	74%	61%	42%
GPT Mini	62%	67%	69%	69%	63%	49%	40%
Gemini	65%	68%	63%	69%	63%	56%	42%
Ollama	45%	50%	59%	50%	46%	40%	28%

Figure 2: F1 scores across all 49 detector-fixer combinations. Rows are detectors; columns are fixers. Red boxes highlight the two peak cells (GPT-5.5 → Opus and GPT-5.5 → GPT-5.5, both 75 %). The row pattern dominates: detector choice determines the ceiling more than fixer choice.

5.3 Per-CWE Detection Performance

Not all CWE categories are equally hard to detect, and the hard ones are exactly the taint-tracing cases where LLMs should have an advantage over static analysis. Table 4 shows detection recall for three representative categories. SSRF (CWE-918) is the starkest: Sonnet detects all 4 cases, while every other model detects at most 2, and GPT-4.1-mini and Ollama detect none. SSRF requires tracing a tainted URL value across function calls to a dangerous sink, a task that demands genuine data-flow reasoning and cannot be solved by pattern matching alone.

ReDoS (CWE-1333) follows a similar pattern but is less extreme: Sonnet, Gemini, and Opus all handle it, while mini and Ollama struggle. SQL injection (CWE-89) inverts the hierarchy entirely: Ollama matches Sonnet at 5/5 while Gemini and Opus get only 3/5, because SQL injection in these cases follows recognizable concatenation patterns that simpler models can catch without deep semantic reasoning.

Table 4: Per-CWE detection recall for selected categories.

CWE	Sonnet	GPT-5.5	Gemini	Opus	Mini	Ollama
CWE-918 SSRF (n=4)	4/4	2/4	1/4	2/4	0/4	0/4
CWE-1333 ReDoS (n=4)	4/4	3/4	4/4	4/4	1/4	0/4
CWE-89 SQL Injection (n=5)	5/5	4/5	3/5	3/5	4/5	5/5

5.4 Detector Quality as the Dominant Variable

To isolate why the detector matters more, consider what false positives cost. A false positive is a case where the detector flags a non-existent vulnerability. The fixer then attempts to patch code that does not need patching. That attempt is scored as a failed fix, directly reducing precision. GPT-5.5 produces 5 false positives across 56 cases; Ollama produces 26. That difference alone accounts for much of the gap between their rows.

Figure 3 plots each detector's false positive count against the best F1 achievable in its row (i.e., with the strongest fixer). The relationship is negative and consistent: detectors with fewer false positives enable higher ceiling F1, regardless of which fixer is chosen.

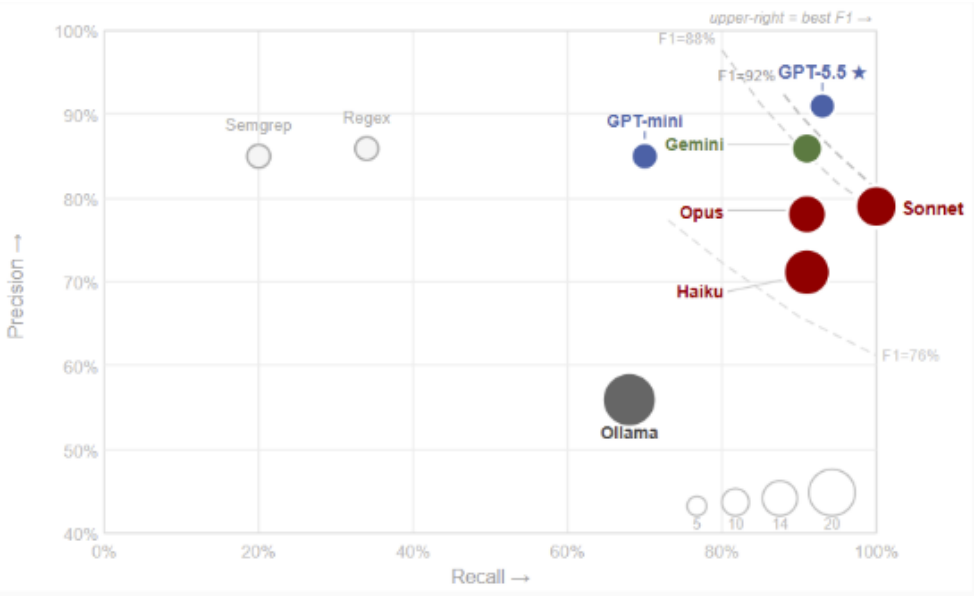

Figure 3: Precision vs. recall for all detectors. Bubble size encodes false positive count. GPT-5.5 achieves the best F1 (92 %) through the highest precision (91 %). Sonnet achieves perfect recall (100 %) but its 17 false positives lower precision to 79 %. Semgrep and Regex fall far to the left, confirming that pattern-based tools miss the majority of cases.

5.5 Ground-Truth Fixer Experiment

To measure pure fixer capability independent of detection noise, each fixer was given a perfect detection: all 56 vulnerabilities identified correctly, zero false positives. Table 5 shows the results. Opus leads at 76 %, fixing 43 of 56 cases. This is 6 percentage points above its real pipeline F1 of 70 %, quantifying exactly how much detection noise costs it. GPT-5.5 fixes 71 % with perfect context, close to its real pipeline F1 of 75 %, consistent with having the cleanest detection in practice.

Ollama failed on all ground-truth cases with errors and is excluded from the table. Even given a

perfect bug description, it cannot reliably produce a correct patch, confirming that its weakness extends beyond detection quality.

Table 5: Fixer performance given perfect detection (56/56 recall, 0 FPs).

Fixer	Fixed	Fix Rate
Opus 4.8	43/56	76%
GPT-5.5	40/56	71%
Haiku 4.5	38/56	67%
Sonnet 4.6	36/56	64%
GPT-4.1-mini	35/56	62%
Gemini 2.5	28/56	50%

5.6 Cost-Efficiency Analysis

Figure 4 plots every pipeline combination by its F1 and its combined detection plus fix cost for 56 cases. The spread is large. The GPT-5.5 self-pipeline costs $7.10 for 75 % F1. The Haiku detector paired with GPT-4.1-mini fixer reaches 68 % F1 for $0.11, a 64× cost reduction for a 7 percentage point drop in F1.

The Pareto frontier makes the trade-off concrete. No pipeline beats 75 % F1, and reaching that level requires a GPT-5.5 detector, which alone costs $2.14 per 56 cases. For teams with a budget constraint, the Haiku-plus-mini combination offers the best F1 per dollar of any paid configuration.

Det↓/Fix→	Sonnet	Opus	Haiku	GPT-5.5	Mini	Gemini	Ollama
Sonnet	$2.09	$1.50	$0.87	$2.38	$0.81	$0.80	$0.76
Opus	$1.74	$2.88	$1.52	$2.81	$1.45	$1.45	$1.41
Haiku	$0.40	$0.81	$0.29	$1.84	$0.11	$0.12	$0.08
GPT-5.5	$2.46	$2.83	$2.25	$7.10	$2.17	$2.19	$2.14
GPT Mini	$0.36	$0.76	$0.16	$1.94	$0.11	$0.09	$0.05
Gemini	$0.41	$0.81	$0.19	$1.84	$0.12	$0.17	$0.09
Ollama	$0.31	$0.70	$0.12	$1.57	$0.03	$0.04	$0.00

Figure 4: Combined detection + fix cost (USD) for all 56 cases across all 49 pipeline combinations. The GPT-5.5 self-pipeline ($7.10) is the most expensive cell. Haiku detect + GPT-4.1-mini fix ($0.11) offers the best value among paid pipelines.

Det↓/Fix→	Sonnet	Opus	Haiku	GPT-5.5	Mini	Gemini	Ollama
Sonnet	32	46	70	29	83	72	46
Opus	39	24	43	25	45	40	28
Haiku	176	87	210	38	620★	489	519
GPT-5.5	29	27	33	11	34	28	20
GPT Mini	171	88	435	36	559	554	800
Gemini	160	84	327	37	535	324	477
Ollama	145	72	508	32	1623	964	free

Figure 5: Fixes per dollar across all 49 pipeline combinations. Higher is better. The $\star$ marks the Haiku detect + GPT-4.1-mini fix cell (620 fixes/$), the best value of any paid configuration. Ollama-fixer cells are free but produce unreliable outputs.

5.7 Ensemble Detection

Rather than picking one detector, an ensemble runs multiple detectors and takes a majority vote: a vulnerability is reported only if at least k of the models flag it. Table 6 shows four configurations.

The frontier majority vote (Sonnet, Opus, GPT-5.5, Gemini; $k \geq 2$) achieves 93 % F1 with 100 % recall and only 8 false positives. The Claude-only ensemble (Sonnet, Opus, Haiku; $k \geq 2$) reaches 90 % F1 and 98 % recall.

Taking the union of all seven models ($k = 1$) collapses to 67 % F1 despite 100 % recall, because every model's false positives are included. The voting threshold is the key lever: too low and false positives dominate; too high and recall drops.

Table 6: Ensemble detection configurations and their detection-only F1.

Strategy	Models	F1	Recall	FPs
Frontier $\geq$2	Sonnet + Opus + GPT-5.5 + Gemini	93%	100%	8
Claude $\geq$2	Sonnet + Opus + Haiku	90%	98%	11
OpenAI $\geq$1	GPT-5.5 + GPT-mini	87%	93%	12
Budget $\geq$1	GPT-mini + Haiku	81%	96%	23
Union all 7	All models ($k = 1$)	67%	100%	54

5.8 Effect of Repeated Scanning

Figure 6 shows detection F1 as a function of the number of times each model scans the same file. Contrary to the intuition that more scans improve coverage, F1 falls as scan count increases for every model. Each additional scan adds false positives faster than it adds true positives, compressing precision. Frontier models (GPT-5.5, Gemini) start high and decline steadily. Ollama starts low and declines further. The practical takeaway is that a single well-calibrated scan outperforms repeated scanning, and investing in a better detector is more effective than running the same detector multiple times.

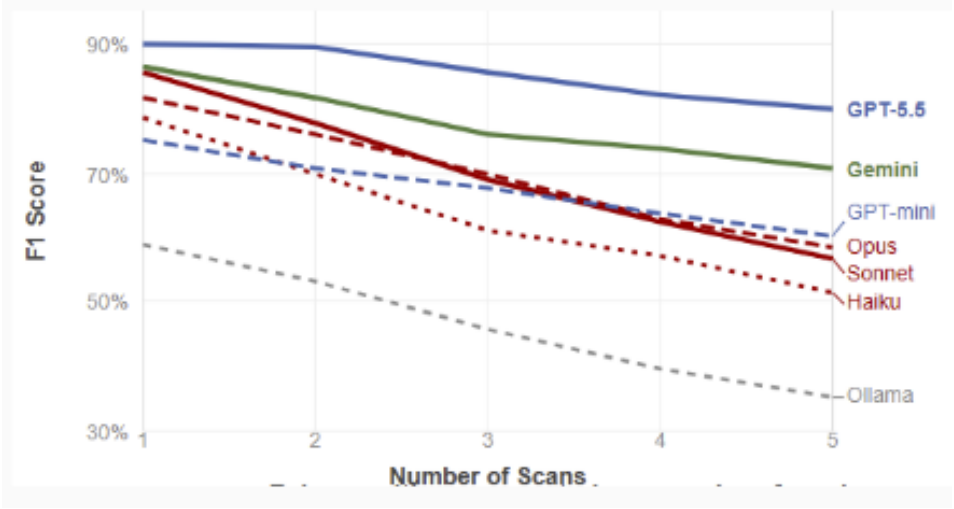

Figure 6: Detection F1 vs. number of scans per case. All models decline as scan count increases because additional scans accumulate false positives faster than they recover missed true positives.

6 Discussion

6.1 Why the Detector Matters More Than the Fixer

The heatmap has a clear structure, and it points in one direction. Vary the fixer while holding the detector fixed: F1 swings by at most 15 percentage points within a row. Vary the detector while holding the fixer fixed: F1 swings by up to 33 points across a column. Detector choice appears to explain roughly twice as much variance in final pipeline F1 as fixer choice, a result that holds consistently across all 49 combinations tested.

The mechanism is not subtle. A false positive is not just a wasted alert; it is an active penalty. The fixer receives the flagged location, attempts to patch code that is not vulnerable, and the oracle scores that attempt as a failed fix. Precision drops. Every false positive costs the pipeline a fix slot it had no chance of filling. The fixer cannot recover from this; it only sees what the detector sends.

The ground-truth experiment puts a number on it. Opus fixes 76 % of cases when given perfect detection and 66 % in the real pipeline. That 10-point gap is attributable to detection noise, not to any weakness in Opus as a fixer. GPT-5.5 tells the opposite story: its detector produces only 5 false positives, so its real-pipeline F1 (75 %) nearly matches its ground-truth fix rate (71 %). The small positive gap is an artifact of F1 formula weighting, not an actual gain from imperfect detection. The takeaway is direct: if a pipeline underperforms, the detector is the more likely culprit.

6.2 Filename Leakage and Model Tier Dependence

Source filenames in the benchmark follow a naming convention that includes the CWE number — for example, `cwe_918_0_js_unsafe.js`. This leaks the vulnerability class directly to the

model before it reads a single line of code. To measure the effect, detection was re-run with generic display names (`code.js`, `code.py`) replacing the original filenames.

Frontier models — Sonnet, GPT-5.5, and Gemini — were unaffected. Their apparent gains between early and late detection rounds trace entirely to four ground-truth label corrections, not to the filename hint. Removing the CWE number from the filename changed nothing for these models; tracing specific missed bugs confirmed that every newly detected case in later rounds was previously mislabeled, not newly discovered.

GPT-4.1-mini lost 8 percentage points when the hint was removed (78% down to 70%), and unlike the frontier models this drop is not explained by label corrections. It simply found fewer bugs without the CWE anchor. This is a meaningful capability distinction: stronger models perform semantic code analysis regardless of what the filename says; weaker models rely on explicit context to focus their search.

For the reported results, all runs use the generic display names. The filename leakage numbers are documented here as a methodological finding rather than a corrected result, because they reveal something true about how model tiers differ in their dependence on prompt scaffolding.

6.3 What Ensemble Detection Buys You

The best solo detector, GPT-5.5, reaches 92 % detection F1. The best ensemble — a majority vote across four frontier models — reaches 93 %. That is a real gain, but a marginal one. Running four models instead of one buys roughly 1 percentage point of detection F1. Whether that trade is worth it depends on context.

The more compelling case for ensembling is recall, not F1. The Claude ensemble (Sonnet, Opus, Haiku; $\geq$2 votes) reaches 98 % recall with only 11 false positives. No solo model achieves both figures simultaneously. For security-critical applications where a missed vulnerability is genuinely more costly than a false alarm (production authentication code, payment processing, anything handling untrusted input at scale), that recall figure matters more than the F1 headline.

The union-of-all strategy shows what happens when ensembling goes wrong. Require only one model to flag a case ($k = 1$ across all seven) and you get 100 % recall but 67 % F1, worse than GPT-5.5 alone. Every model's false positives accumulate, and the combined noise overwhelms the signal. The voting threshold is what makes ensemble detection viable; without it, you have just aggregated the weaknesses of every model in the pool.

6.4 Practical Pairing Recommendations

Three configurations stand out, depending on budget and risk tolerance.

For maximum F1, GPT-5.5 as detector paired with Opus as fixer reaches 75 % at a combined cost

of $2.83 per 56 cases. GPT-5.5 produces only 5 false positives across the benchmark, the cleanest detection of any model tested, which gives Opus unusually clean inputs. Opus, in turn, leads all fixers at 76 % when given perfect detection. No other combination in the heatmap matches this F1.

For cost-sensitive deployments, the picture changes considerably. Haiku detection paired with GPT-4.1-mini fixing reaches 68 % F1 for $0.11 total, roughly 26 times cheaper than the top configuration for a 7 percentage point drop. Haiku's 94 % detection recall is high enough to catch the large majority of real vulnerabilities, and mini fixes more than half of what it receives. That is a defensible trade for teams where cost is a genuine constraint.

For teams operating entirely within Anthropic's model family, Haiku detection with Opus fixing reaches 71 % F1. It costs more than the budget pair but avoids cross-provider API dependencies and maintains strong recall.

Ollama, at the other end, should not be used in either role for production-grade remediation. Its 68 % detection recall and 26 false positives make it the weakest detector tested, and its fixer fails entirely on the ground-truth task. The gap between Ollama and the next-weakest model is larger than the gap between that model and the best.

7 Threats to Validity

7.1 Construct Validity

F1 is computed from a binary oracle verdict: a case is either fixed correctly or it is not. That is a practical choice, but it is not a complete picture of security quality. A patch that passes the oracle may still leave the codebase in a worse state than before: it could suppress the specific exploit the oracle tests while leaving a related weakness intact, or it could introduce a subtle logic error that no test in the suite exercises. The oracle reduces that risk by combining a functionality check with an exploit-based security check for CWEval cases, but it cannot guarantee that a passing patch is fully correct in all contexts.

False positives are also counted at the case level, not the finding level. A detector that flags three lines in a single case contributes one false positive to the F1 calculation, not three. This slightly understates the noise burden on the fixer for cases where the detector is particularly verbose.

7.2 Internal Validity

Each detector-fixer combination was run once. LLM outputs are non-deterministic: the same prompt with the same model can produce different outputs across calls, especially for borderline cases. The results in Section 5 are point estimates, not averages over repeated runs. Differences of one or two cases (roughly 2–4 percentage points on a 56-case benchmark) should be treated as within the noise margin rather than as reliable distinctions.

7.3 External Validity

The benchmark covers 56 cases across 23 CWE categories, in JavaScript and Python only. The results may not transfer to other languages, to vulnerabilities that span multiple files, or to cases where the fix requires architectural changes rather than a localized code edit. All cases were selected to have self-contained, single-file vulnerabilities precisely to keep the evaluation controlled, but that selection criterion also means the benchmark favors the conditions where LLM fixers are strongest.

The models evaluated represent a snapshot of capability as of mid-2026. Relative rankings between models may shift as newer versions are released, and the absolute F1 numbers will likely improve as model capability increases. The structural finding, that detector false positive rate dominates pipeline F1 more than fixer capability, is more likely to generalize than any specific number.

7.4 Reproducibility

All detection outputs are stored as versioned JSONL files in the repository. The `CachedFindingDetector` replays them identically, so any fixer column can be re-run without re-querying the detection models. The CWEval oracle runs in Docker with a pinned image, which makes the functionality and security checks deterministic across machines. Seeded and literature cases use native test suites committed alongside the source, so those verdicts are also fully reproducible without external dependencies.

8 Future Work

The most immediate limitation is the single-run design. Running each detector-fixer combination three or more times would produce confidence intervals and make it possible to distinguish real performance differences from run-to-run variance. The harness already supports a `repeats` parameter; the barrier is cost, not infrastructure.

Language coverage is the next natural expansion. JavaScript and Python share similar vulnerability patterns and both benefit from the same style of local-semantic reasoning. Adding Rust, Go, Java, or C/C++ would test whether the detector quality advantage holds across languages with different memory models, type systems, and idiomatic security patterns. C and C++ in particular would introduce memory safety vulnerabilities that the current benchmark does not cover at all.

Multi-file vulnerabilities are a harder and more realistic challenge. Every case in the current benchmark is self-contained in a single file. Real codebases frequently have vulnerabilities where the source and the sink are in separate modules, or where a fix requires changing an interface definition and all its callers. LLM fixers that work well on single-file cases may degrade significantly on cross-file tasks, and the CachedFindingDetector design would need to be extended to support multi-file context windows.

Multi-turn fixing is another direction. The current fixer makes one attempt and receives a binary

verdict. A fixer that could read the oracle's failure output and try again would more closely resemble how a human developer iterates on a patch. This would also make the benchmark more directly comparable to agent-based repair systems that run in a loop.

Finally, the cost analysis does not account for prompt caching. Several providers offer significant discounts on repeated prompt prefixes, which would disproportionately benefit the ensemble detection strategy since the same source file is sent to multiple detectors. A caching-aware cost model could change the cost-efficiency frontier meaningfully.

9 Conclusion

This project set out to answer two questions: whether LLMs can match static analysis for vulnerability detection, and whether composing a strong detector with a strong fixer produces better outcomes than either model alone. The answer to both is yes, with important caveats.

On detection, frontier LLMs clearly outperform the regex-based static analysis baseline on the local-semantic cases that make up the majority of the benchmark. GPT-5.5 reaches 91 % detection F1 solo, and a four-model frontier ensemble reaches 92 %. Static analysis catches syntactic patterns reliably but cannot reason about data flow, making it structurally weaker on the class of vulnerabilities that requires understanding what a value is and where it came from.

On composition, the 7×7 heatmap shows that pairing a strong detector with a capable fixer does improve outcomes over the weakest pairings, but the gain is asymmetric. The detector drives the result. Replacing a weak detector with a strong one in a fixed column improves F1 by up to 33 percentage points. Replacing a weak fixer with a strong one in a fixed row improves F1 by at most 15. The practical recommendation follows directly: invest in detection quality before optimizing the fixer.

The cost analysis reveals that the best accuracy does not require the most expensive pipeline. Haiku detection with GPT-4.1-mini fixing reaches 68 % F1 for $0.11 per 56 cases, roughly 64 times cheaper than the GPT-5.5 self-pipeline for a 7 point difference in F1. For most deployment scenarios, that trade-off is worth taking.

Two findings stand out as the most transferable beyond this specific benchmark. First, false positive rate in detection is more important than fixer capability for final pipeline F1, a result that holds consistently across all 49 combinations tested. Second, the Ollama 7b model is not yet viable for either role in a production remediation pipeline, with a 33 percentage point gap separating it from the next-weakest model.

References

Manish Bhatt, Sahana Chennabasappa, Cyrus Nikolaidis, Shengye Wan, Ivan Evtimov, Dominik Gabi, Daniel Song, Faizan Ahmad, Cornelius Aschermann, Lorenzo Fontana, Sasha Frolov, Ravi Prasad Giri, Dhaval Kapil, Chrisoph Kozyrakis, David LeBlanc, James Milazzo, Andres Straumann, Gabriel Synnaeve, Varun Vontimitta, Spencer Whitman, and Joshua Saxe. CyberSecEval: A benchmark for evaluating the cybersecurity risks of large language models. *arXiv preprint arXiv:2312.04724*, 2023.

Michael Fu and Chakkrit Tantithamthavorn. LineVul: A transformer-based line-level vulnerability prediction. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2022.

Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. Impact of code language models on automated program repair. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Zheng Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 20(2), 2023.

Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.

Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. Examining zero-shot vulnerability repair with large language models. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.

Jinjun Peng et al. CWEval: Outcome-driven evaluation on functional and security quality of code agents. *arXiv preprint arXiv:2501.08200*, 2025.

Mohammed Latif Siddiq and Joanna C. S. Santos. SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques. In *International Workshop on Mining Software Repositories for Security (MSR4P&S)*, 2022.

Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of ChatGPT. In *IEEE/ACM International Workshop on Automated Program Repair (APR)*, 2023.

Chandra Thapa, Seyit Camtepe, Surya Nepal Chhetri, and Seyit A. Camtepe. Transformer-based

language models for software vulnerability detection. In *Annual Computer Security Applications Conference (ACSAC)*, 2022.

Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

A Full Heatmap Data

Table 7: F1 (%) for all 49 detector-fixer combinations (v2 methodology).

Det \ Fix	Sonnet	Opus	Haiku	GPT-5.5	Mini	Gemini	Ollama
Sonnet	67	69	61	70	67	58	35
Opus	68	70	65	70	65	58	40
Haiku	70	71	62	69	68	60	42
GPT-5.5	72	75	74	75	74	61	42
Mini	62	67	69	69	63	49	40
Gemini	65	68	63	69	63	56	42
Ollama	45	50	59	50	46	40	28